

Authored by: Arpit Raj, LNMIIT Jaipur

July 30, 2026

LC 1376: TIME NEEDED TO INFORM ALL EMPLOYEES

Google, Amazon

Problem Understanding

- Given $n \rightarrow$ emp from 0 to $n-1$
- $headID \rightarrow$ source of news
- a vector of $manager\ ID$
- a vector of $inform\ time$

Examples & Visual Walkthrough

Example 1:

Tree structure: Node 2 is connected to nodes 0, 1, 3, 4, 5

- $n = 6$
- $head = 2$
- $manager = [2, 2, -1, 2, 2, 2]$ (indices 0 1 2 3 4 5)
- $informTime = [0, 0, 7, 0, 0, 0]$

op = 7 (time took by 2 to distribute the news)

Example 2:

Tree structure: 0 connects to 1, 2. Node 1 connects to 3, 4. Node 2 connects to 5.

- $informTime = [2, 3, 4, 0, 0, 0]$ (indices 0 1 2 3 4 5)

So at 2nd minute news reached 1, 2

at (2+3)rd minute news reached 1, 2, 3, 4

at (2+4)rd minute news reached 1, 2, 3, 4, 5

so $2 + \max(child) \rightarrow 2 + \max(3, 4) = 6$ minutes

Core Idea & Approach

- Graph given is $\rightarrow$ employee $\rightarrow$ manager
- To do DFS we need $\rightarrow$ manager $\rightarrow$ employee

Core Idea $\rightarrow$ Take max time among children.

Inform every child.

Child continues informing.

Let current employee = **curr**

For every subordinate $\rightarrow$ **dfs(child)**

return **informTime[curr] + max(child)**

Algorithm Steps

S1: Make manager to employee adjacency list

S2: Start dfs from headID

S3: dfs needs adjacency list, informTime, current employee

S4: Base case $\rightarrow$ if leaf node then return 0 [it doesnt need to pass on info]

S5: return **informTime[curr] + max child time**

Implementation (C++)

```
class Solution {
public:

    int dfs(vector<vector<int>>& adj, vector<int>& t, int curr) {

        //agar leafnode he to inform krne ko koi nhi
        if(adj[curr].size() == 0) return 0;

        int maxu = 0;
        for(auto &it : adj[curr]) { //har ek child me jaake dekho slowest kon he

            maxu = max(maxu, dfs(adj, t, it));

        }

        return t[curr] + maxu;
    }

    int numOfMinutes(int n, int headID, vector<int>& manager, vector<int>& informTime) {
        //basic idea ye rkhenge ki
        //manager to child dfs kreng
        //time taken will be current ka time + slowest child ka time
        // eg: 5 aur 7 minute ke do branch , saare employee ko to 7 minute baad hi p
        //no matter how fast was 5 minute wala branch

        //sbse pehle manager to employee adjacency list

        vector<vector<int>> adj(n);
        for(int i = 0; i < n; i++) {
            if(manager[i] == -1) continue;
            adj[manager[i]].push_back(i);
        }

        return dfs(adj, informTime, headID);
    }
};
```